Q7 – Diagnose, Redesign and Validate

Selected bottleneck

A suitable redesign case is counting word frequencies by calling list.count() for every distinct word. Each count scans the complete list, so n words and d distinct words require O(n·d) work, approaching $O(n^2)$ when d grows with n.

Objective and revised design

Objective: return identical frequencies while reducing repeated scans. Use one pass with Counter or a dictionary. Each word performs one average O(1) dictionary update, giving O(n) expected time and O(d) storage.

Prediction and validation

Predict a larger speed-up as input size and distinct-word count grow. The supplied program compares the repeated scan with Counter using the same generated input and verifies correctness on a sample. Record measured times on the execution machine.

Trade-off and defence

The one-pass method stores one counter per distinct word, while a repeated scan may use less auxiliary state but performs much more work. For tiny inputs, the simple scan can be adequate; for large datasets, the linear pass is preferable. The benchmark output, not invented values, should support the final claim.

Python Program

```
from time import perf_counter
from collections import Counter
import random

def count_repeated_scan(words):
    return {w: words.count(w) for w in set(words)}

def count_once(words):
    return dict(Counter(words))

def compare(n=100_000):
    random.seed(7)
    words = [f"w{random.randrange(1000)}" for _ in range(n)]
    for fn in (count_repeated_scan, count_once):
        start = perf_counter()
        result = fn(words)
        elapsed = perf_counter() - start
        print(fn.__name__, len(result), f"{elapsed:.6f}s")
    assert count_repeated_scan(words[:500]) == count_once(words[:500])

if __name__ == "__main__":
    compare()
```